

PRODUCT REQUIREMENTS DOCUMENT

Intelligent Integration Engine

Office of the Head of Civil Service (OHCS), Ghana

Event-Driven Process Intelligence Platform

Author: **Ozzy** — **OHWP Studios**

Date: July 2026 | Version 1.0

Stack: Cloudflare Workers · D1 · R2 · Queues · Workers AI · Pages

Build Tool: [Kimi Code](#)

1. Executive Summary

This PRD defines the Intelligent Integration Engine (IIE) — a platform that connects the OHCS AI Chatbot, Attendance Management System, and Leave Approval Workflow through a unified, event-driven architecture hosted entirely on Cloudflare's edge infrastructure.

The engine is not about the individual apps. It's about the integration layer that captures every organizational event into a Unified Event Log, then applies process mining to discover real workflows, detect bottlenecks, check conformance, and generate actionable decision-support intelligence.

Building on Cloudflare means zero-server ops, global edge performance, and a generous free tier that keeps costs near zero during development and early deployment.

2. Problem Statement

OHCS currently manages attendance, leave approvals, and HR inquiries through disconnected manual or semi-digital processes. This creates three core problems:

- No cross-system visibility — a leave request can't reference an employee's attendance pattern because the systems don't talk to each other.
- No process intelligence — managers rely on anecdotal evidence to identify bottlenecks; there's no data-driven view of how workflows actually execute.
- No unified audit trail — events are scattered across spreadsheets, paper forms, and ad-hoc databases, making conformance checking impossible.

3. Goals & Non-Goals

Goals

- Build a central Integration Engine that connects all three subsystems via standardized event APIs.
- Capture every organizational event into a Unified Event Log with a canonical schema (case_id, activity, resource, timestamp, source_system, metadata).
- Apply process mining (workflow discovery, bottleneck detection, conformance analysis) on the event log.
- Deliver a decision-support dashboard with actionable recommendations.
- Deploy the entire stack on Cloudflare's edge platform — zero traditional servers.

Non-Goals

- Replacing existing HR/payroll systems — the IIE integrates alongside them.
- Biometric hardware procurement — RFID/card reader selection is out of scope; the system consumes attendance events via API.
- Mobile native apps — the MVP is web-first (PWA-capable via Cloudflare Pages).

4. System Architecture

The platform follows a layered, event-driven architecture deployed entirely on Cloudflare's edge.

4.1 Architecture Layers

Layer	Responsibility	Cloudflare Service
Presentation	Dashboard UI, Chatbot interface, Admin panels	Pages (static + SPA)
API Gateway	Route requests, auth, rate limiting	Workers + Workers AI (auth)
Integration Engine	Event ingestion, normalization, routing	Workers + Queues
Event Store	Unified Event Log persistence	D1 (SQLite at edge)
Process Intelligence	Mining, bottleneck detection, conformance	Workers (scheduled CRON)
Storage	Documents, reports, exports	R2 (S3-compatible)
AI Layer	Chatbot NLP, anomaly detection, recommendations	Workers AI (LLM + embeddings)

4.2 Event Flow

Every subsystem publishes events to the Integration Engine via a POST /api/events endpoint. The flow is:

- Subsystem generates an event (e.g., employee clocks in, leave request submitted, chatbot query).
- Worker validates the event payload against the canonical schema.
- Event is written to D1 (Unified Event Log) and published to a Cloudflare Queue.
- Queue consumers trigger downstream logic: update dashboards, run conformance checks, feed process mining batch jobs.
- CRON-triggered Workers run process mining algorithms on the event log periodically (e.g., every 6 hours).

4.3 Canonical Event Schema

Field	Type	Description
event_id	UUID	Unique event identifier (auto-generated)
case_id	STRING	Groups events into a process instance (e.g., leave request ID)
activity	STRING	What happened: 'clock_in', 'leave_submitted', 'hr_approved'
resource	STRING	Who/what performed it: employee ID, system name
timestamp	ISO 8601	When it occurred
source_system	ENUM	CHATBOT ATTENDANCE LEAVE_WORKFLOW
metadata	JSON	Arbitrary payload (department, location, etc.)

5. Subsystem Specifications

5.1 AI Chatbot

Runtime: Cloudflare Worker + Workers AI (LLM model)

Interface: Embedded chat widget on the dashboard (Pages)

The chatbot is the natural-language front door. Employees ask questions and the chatbot routes to the correct subsystem API.

Capabilities

- Answer HR policy questions using RAG (policy docs stored in R2, embeddings via Workers AI).
- Query attendance records: "How many days was I absent this month?"
- Check leave balance and initiate leave requests via conversational flow.
- Every chatbot interaction emits an event to the Integration Engine.

5.2 Attendance Management System

Runtime: Cloudflare Worker (API) + D1 (attendance records)

Input: RFID/access card readers POST attendance events via webhook

Capabilities

- Receive clock-in / clock-out events from hardware readers.
- Calculate daily attendance summaries per employee and department.
- Flag anomalies: late arrivals, early departures, missing clock-outs.
- Emit standardized events: clock_in, clock_out, anomaly_detected.

5.3 Leave Approval Workflow

Runtime: Cloudflare Worker (state machine) + D1 (workflow state) + Queues (step transitions)

Workflow Steps

- Employee submits request → Line Manager reviews → HR verifies → Director approves → Employee notified.
- Each transition emits an event with the previous and next state.
- Supports: escalation (auto-escalate if no action in 48h), delegation, rejection with reason, and cancellation.
- Durable Objects (optional) can hold per-request state for real-time status queries.

6. Process Intelligence Module

This is the research core. The module operates on the Unified Event Log to produce organizational intelligence.

6.1 Workflow Discovery

Using the Alpha Miner or Heuristic Miner algorithm (implemented in a Worker), the engine reconstructs actual process models from event sequences. This reveals how work really flows vs. how policy says it should.

Implementation

- A scheduled CRON Worker queries the event log, groups events by case_id, orders by timestamp, and builds a directly-follows graph.
- The graph is stored in D1 and visualized on the dashboard as a process map (rendered client-side with D3.js or similar).

6.2 Bottleneck Detection

For each workflow step transition, the engine calculates median and P95 duration. Steps exceeding a configurable threshold are flagged.

- Example insight: "HR Verification averages 4.2 days — 3x longer than any other step in leave approval."
- Bottleneck data is surfaced on the dashboard with drill-down by department, time period, and approver.

6.3 Conformance Checking

The engine compares discovered workflows against the prescribed process model (defined in configuration). Deviations are logged.

- Example: "12 leave requests bypassed Line Manager approval and went directly to HR."
- Conformance scores are computed per process and trended over time.

6.4 Decision Support

The engine generates recommendations by combining bottleneck data, conformance violations, and attendance patterns:

- "Department X has 23% higher approval delays — consider adding a backup approver."
- "Monday attendance dips 15% in Q1 — correlates with increased leave requests filed on Fridays."
- "3 workflow variants account for 80% of cases — consider simplifying the other 7."

7. Technology Stack & Cloudflare Mapping

Component	Technology	Why
Frontend	React/Next.js on Cloudflare Pages	Static export + edge SSR; instant global deploys
API Layer	Cloudflare Workers (TypeScript)	0ms cold start, 100k+ free requests/day, edge-native
Database	Cloudflare D1 (SQLite)	Serverless SQL at edge; perfect for event log + relational queries
Message Queue	Cloudflare Queues	Decouple event ingestion from processing; guaranteed delivery
Object Storage	Cloudflare R2	S3-compatible, zero egress fees; stores docs, exports, policy PDFs
AI/ML	Workers AI	Run LLMs + embeddings at edge for chatbot; no external API costs
Auth	Cloudflare Access / custom JWT via Worker	Zero-trust auth; integrates with OHCS SSO if available
Caching	Cloudflare KV	Global key-value for session data, config, and hot queries
Scheduling	Workers CRON Triggers	Run process mining jobs on schedule (every 6h, daily)
IDE/Build	Kimi Code	AI-assisted coding for rapid Worker + React development

8. How I'd Build It — Architecture Vision

If I were building this from scratch with Cloudflare + Kimi Code, here's the exact approach I'd take, phase by phase.

Phase 1: Foundation (Weeks 1–3)

Goal: Event backbone + skeleton API

- Scaffold a Cloudflare Workers monorepo using Wrangler. One Worker per domain: api-gateway, event-engine, attendance, leave-workflow, chatbot.
- Define the D1 schema: events table (the Unified Event Log), employees, departments, leave_requests, attendance_records.
- Build POST /api/events — the single ingestion endpoint. Validate against the canonical schema, write to D1, publish to a Queue.
- Deploy to a staging *.workers.dev subdomain. Seed with synthetic events using a test script.
- Use Kimi Code to generate boilerplate: Worker endpoints, D1 migrations, TypeScript types from the event schema.

Phase 2: Subsystems (Weeks 4–7)

Goal: Three working subsystems emitting real events

- Attendance Worker: POST /api/attendance/clock-in and /clock-out. Validates employee ID, writes attendance record, emits event. Add a CRON job for daily anomaly detection.
- Leave Workflow Worker: Implements a state machine (submitted → manager_review → hr_verification → director_approval → completed/rejected). Each transition emits an event. Queues handle async notifications (email via Cloudflare Email Workers or a simple webhook).
- Chatbot Worker: Workers AI with a system prompt grounding it in OHCS context. RAG pipeline: policy docs chunked and embedded in R2 + vectorize (or a simple KV-based embedding store). Routes action intents to attendance/leave APIs.

Phase 3: Process Intelligence (Weeks 8–10)

Goal: Mining algorithms running on the event log

- Implement a Heuristic Miner in TypeScript (runs inside a Worker). It queries D1 for events, groups by case_id, builds a directly-follows graph with frequency/dependency scores.
- Bottleneck calculator: for each activity pair, compute median/P95/max duration. Flag outliers.
- Conformance checker: load the prescribed model from KV config, compare against discovered traces, score each case.
- All results written to D1 analysis tables. CRON triggers these every 6 hours.

Phase 4: Dashboard & UX (Weeks 11–14)

Goal: Visual intelligence layer

- React app on Cloudflare Pages. Tailwind CSS for styling. Three main views:
 - Operations Dashboard — live event feed, attendance heatmap, leave pipeline status.
 - Process Intelligence — interactive process map (D3.js), bottleneck overlay, conformance score trends.
 - Decision Support — AI-generated insights cards, department comparisons, recommendation feed.
- Embedded chatbot widget in the bottom-right corner.
- Auth via Cloudflare Access (or simple JWT auth if Access isn't available).

Phase 5: Polish & Research Validation (Weeks 15–16)

- Load test with realistic event volumes (e.g., 500 employees, 6 months of synthetic data).
- Validate process mining output against known process models.
- Write up findings: discovered workflows vs. prescribed, bottleneck identification accuracy, conformance deviation rates.
- Optimize D1 queries (indexes on case_id, timestamp, source_system).

9. D1 Database Schema (Key Tables)

Table	Purpose	Key Columns
events	Unified Event Log	event_id, case_id, activity, resource, timestamp, source_system, metadata
employees	Staff directory	employee_id, name, department_id, role, email, card_id
departments	Org structure	department_id, name, head_employee_id
attendance_records	Daily attendance	record_id, employee_id, date, clock_in, clock_out, status
leave_requests	Leave state machine	request_id, employee_id, type, start_date, end_date, status, current_step
workflow_transitions	Leave step history	transition_id, request_id, from_step, to_step, actor_id, timestamp
process_models	Discovered workflows	model_id, source, graph_json, created_at
bottlenecks	Detected delays	id, activity_pair, median_ms, p95_ms, flagged, period
conformance_results	Deviation logs	id, case_id, deviation_type, description, score, detected_at

10. API Endpoints (Core)

Method	Endpoint	Description
POST	/api/events	Ingest a new event into the Unified Event Log
GET	/api/events?case_id=X	Retrieve event trace for a specific case
GET	/api/events/stream	SSE stream of real-time events (for dashboard)
POST	/api/attendance/clock-in	Record employee clock-in
POST	/api/attendance/clock-out	Record employee clock-out
GET	/api/attendance/:employee_id/summary	Attendance summary for an employee
POST	/api/leave/request	Submit a new leave request
POST	/api/leave/:id/transition	Advance leave workflow to next step
GET	/api/leave/:id/status	Current status of a leave request
POST	/api/chatbot/message	Send a message to the AI chatbot
GET	/api/intelligence/process-map	Get discovered process model graph
GET	/api/intelligence/bottlenecks	Get current bottleneck analysis
GET	/api/intelligence/conformance	Get conformance check results
GET	/api/intelligence/recommendations	Get AI-generated recommendations

11. Cost Estimate (Cloudflare Free + Paid Tiers)

One of the biggest advantages of Cloudflare is the generous free tier. For a research project / initial deployment, most of this runs free.

Service	Free Tier	Estimated Monthly Cost (MVP)
Workers	100,000 requests/day	\$0 (well within free tier for OHCS scale)
D1	5M rows read, 100K writes/day	\$0 (sufficient for hundreds of employees)
Queues	1M operations/month	\$0
R2	10GB storage, 10M reads	\$0 (policy docs + exports fit easily)
Workers AI	10,000 neurons/day	\$0 for light chatbot use; ~\$5/mo at scale
Pages	500 builds/month, unlimited bandwidth	\$0
KV	100K reads/day	\$0
Total (MVP)		\$0–\$5/month

12. Risks & Mitigations

Risk	Impact	Mitigation
D1 row limits at scale	High	Partition event log by month; archive old events to R2 as Parquet
Workers AI model quality	Medium	Fall back to external LLM API (OpenAI/Anthropic) if edge models insufficient
RFID hardware integration	Medium	Define a simple webhook contract; hardware team implements the POST
Process mining complexity in Workers	Medium	Keep algorithms simple (Heuristic Miner); offload heavy analysis to scheduled batch jobs
User adoption at OHCS	High	Involve end-users in dashboard design; chatbot lowers barrier to entry

13. Success Metrics

- 100% of subsystem events captured in the Unified Event Log with <500ms ingestion latency.
- Process mining discovers at least 3 distinct workflow variants from real operational data.
- Bottleneck detection identifies the slowest approval step with >90% accuracy vs. manual audit.
- Conformance checker flags >80% of known policy violations in test scenarios.
- Dashboard loads in <2 seconds globally (edge deployment advantage).
- Chatbot resolves >60% of HR policy queries without human escalation.

14. Timeline Summary

Phase	Duration	Deliverable
1 — Foundation	Weeks 1–3	Event backbone, D1 schema, ingestion API
2 — Subsystems	Weeks 4–7	Chatbot, Attendance, Leave Workflow — all emitting events
3 — Intelligence	Weeks 8–10	Process mining, bottleneck detection, conformance checking

Phase	Duration	Deliverable
4 — Dashboard	Weeks 11–14	Full UI with process maps, insights, embedded chatbot
5 — Validation	Weeks 15–16	Load testing, research validation, documentation

Total estimated timeline: 16 weeks (4 months) for a fully functional MVP with process intelligence capabilities.